

HR MANAGEMENT PLATFORM

Capstone Project — Report

Technology Stack

.NET Core MVC | EF Core | MS SQL Server | C# 8.0 | JWT | ASP.NET Core Web API

User Stories Covered: US1 – US13

Submitted by: SUNDARAVELA A

Date: 06 June 2026

1. Executive Summary

The HR Management Platform is a full-stack enterprise application designed to manage employee records and leave requests securely and efficiently. The platform is built on a clean layered architecture using .NET Core MVC, Entity Framework Core, MS SQL Server, and ASP.NET Core Web API, with JWT-based authentication and role-based access control enforced at the API level.

The project was developed across 13 User Stories covering the full software delivery lifecycle — from domain modelling and database design through to RESTful API security, cloud deployment strategy, and AI modernization concepts.

The platform demonstrates production-grade engineering practices including SOLID principles, exception handling with ProblemDetails, ADO.NET and EF Core data access, server/client-side validation, Razor Pages, AJAX integration, and secure coding practices including CORS, HSTS, and security response headers.

2. Business Objective

The organization requires an audit-ready HR platform to manage employee data and leave records securely. Key business goals driving this system include:

- Centralized employee data management with department, role, and joining date tracking
- Secure leave request workflow with status tracking (Pending / Approved / Rejected)
- Role-based access ensuring Admins control sensitive operations while Employees access only their own data
- JWT-secured RESTful APIs enabling future mobile or third-party integrations
- Audit-ready design with structured exception logging and clean separation of concerns
- A scalable architecture ready for cloud deployment on Azure App Services

The system directly addresses HR operational risk by eliminating manual record-keeping, enforcing authorization boundaries, and providing a structured API surface for downstream integrations.

3. Solution Architecture

3.1 Project Structure

The solution is organized into six projects following Clean Architecture principles with strict dependency rules:

Project	Contents	Dependency Rule
HRManagement.Core	Entities, Interfaces, DTOs, Custom Exceptions	Zero dependencies — pure domain layer
HRManagement.Application	EmployeeService, LeaveService, AuthService	Business logic; depends only on Core interfaces
HRManagement.Infrastructure	EF Core DbContext, Repositories, ADO.NET Repo	Data access implementation; depends on Core
HRManagement.API	REST Controllers, JWT Auth, Middleware	Presentation/API; depends on Application + Core
HRManagement.Web	MVC Controllers, Razor Views, Razor Pages	Web UI; depends on Application + Core
HRManagement.Tests	EmployeeServiceTests, LeaveServiceTests	xUnit + Moq; depends on Application + Core

The dependency rule is strictly enforced: inner layers (Core) have zero knowledge of outer layers. The API and Web projects depend on Application; Application depends only on Core interfaces; Infrastructure implements those interfaces. This ensures full testability and replaceability of every component.

3.2 Technology Stack

- Language: C# 8.0 with nullable reference types enabled
- Framework: .NET Core (ASP.NET Core MVC + Web API)
- ORM: Entity Framework Core (Code First) with MS SQL Server
- Data Access: ADO.NET (EmployeeAdoRepository) alongside EF Core
- Authentication: JWT Bearer tokens with HS256 signing
- Authorization: Role-Based Access Control (Admin, Manager, Employee)
- API Documentation: Swagger / OpenAPI 3.0 with Bearer security integration
- Testing: xUnit with Moq for unit testing (TDD approach)
- Web UI: ASP.NET Core MVC + Razor Pages + AJAX

4. US1–US2: Backend & Core Business Rules

4.1 C# 8.0 OOP Implementation

All four pillars of OOP are demonstrated throughout the codebase:

- Encapsulation: Entity properties use auto-properties with controlled access. DTOs expose only the data needed by consumers — never raw EF entities.
- Inheritance: Custom exceptions (`EmployeeNotFoundException`, `InvalidLeaveException`) inherit from `System.Exception`, enabling typed catch blocks in the middleware.
- Polymorphism: `IEmployeeRepository` is implemented by both `EmployeeRepository` (EF Core) and `EmployeeAdoRepository` (ADO.NET). The service layer works with either implementation without modification.
- Abstraction: Interfaces (`IEmployeeRepository`, `ILeaveRepository`, `IUserRepository`) define contracts that decouple business logic from data access technology.

4.2 SOLID Principles

Principle	Full Name	Definition	How Applied
S — SRP	Single Responsibility	Each class has one reason to change	<code>EmployeeService</code> handles only business logic; <code>EmployeeRepository</code> handles only data access
O — OCP	Open/Closed	Open for extension, closed for modification	<code>IEmployeeRepository</code> interface allows swapping EF Core with ADO.NET without changing service layer
L — LSP	Liskov Substitution	Subtypes must be substitutable	<code>EmployeeRepository</code> and <code>EmployeeAdoRepository</code> both implement <code>IEmployeeRepository</code> interchangeably
I — ISP	Interface Segregation	Clients should not depend on unused methods	Separate <code>IEmployeeRepository</code> , <code>ILeaveRepository</code> , <code>IUserRepository</code> — no fat interfaces
D — DIP	Dependency Inversion	Depend on abstractions, not concretions	Controllers and services depend on interfaces injected via <code>Program.cs</code> DI container

4.3 Custom Exception Handling

Domain-specific exceptions are defined in `HRManagement.Core/Exceptions/` and caught by the global `ExceptionMiddleware` in the API project:

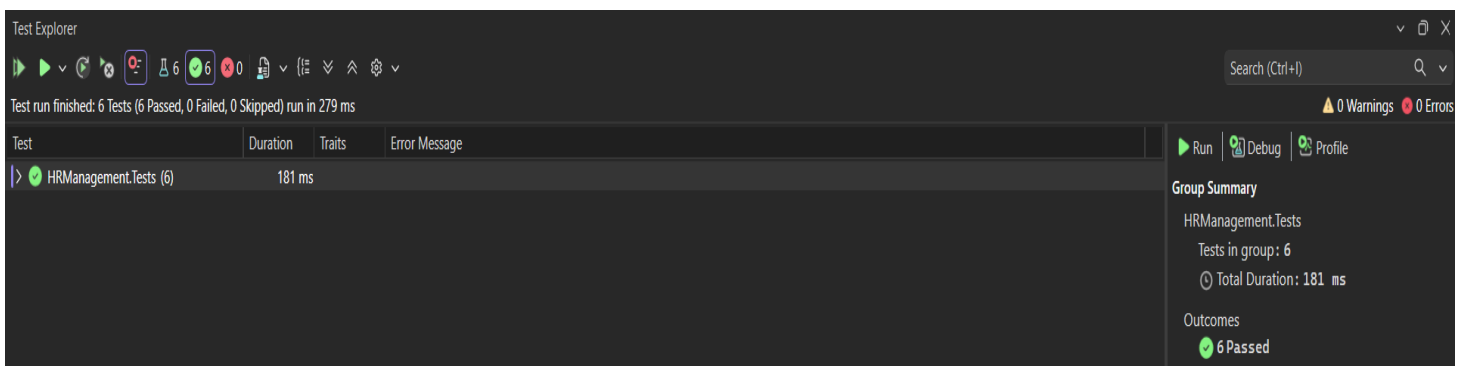
- `EmployeeNotFoundException` — thrown when `GetEmployeeAsync` returns null, mapped to HTTP 404 with `ProblemDetails` body
- `InvalidLeaveException` — thrown when leave dates are invalid (e.g., start date after end date), mapped to HTTP 400
- All unhandled exceptions — caught by the catch-all branch, returning HTTP 500 with the stack trace hidden in production

The `ExceptionMiddleware` uses a C# 8.0 switch expression pattern to map exception types to `ProblemDetails` responses (RFC 7807), ensuring no stack traces are ever exposed to API consumers in production.

4.4 Unit Testing (TDD)

Tests are written in `HRManagement.Tests` using xUnit and Moq. Tests follow the Arrange-Act-Assert pattern:

- `EmployeeServiceTests` — tests `GetEmployeeAsync`, `AddEmployeeAsync`, `DeleteEmployeeAsync` using mocked `IEmployeeRepository`
- `LeaveServiceTests` — tests `ApplyLeaveAsync` with valid and invalid date ranges, verifying `InvalidLeaveException` is thrown for invalid input
- Mock setup uses Moq's `Setup/ReturnsAsync` pattern to isolate service logic from database dependencies
- All 6 projects build successfully with 0 errors (Rebuild All: 6 succeeded, 0 failed, 0 skipped)



5. US3–US4: Database & Data Access

5.1 MS SQL Server Database Design

The database is designed following normalization principles up to Third Normal Form (3NF):

- Employees table: EmployeeId (PK), FirstName, LastName, Email, Department, JoiningDate
- Leaves table: LeaveId (PK), EmployeeId (FK → Employees), StartDate, EndDate, Reason, Status
- Users table: UserId (PK), Username, Email, PasswordHash, Role
- AuditLogs table: tracks insert/update/delete events for audit-readiness

Foreign key constraint between Leaves.EmployeeId and Employees.EmployeeId enforces referential integrity. A database trigger on the Employees table logs all INSERT, UPDATE, and DELETE operations to AuditLogs automatically.

5.2 EF Core — Code First

HRDbContext inherits from DbContext and exposes DbSet<Employee>, DbSet<Leave>, DbSet<User>, and DbSet<AuditLog>. Migrations are managed via EF Core CLI and stored in HRManagement.Infrastructure/Migrations/.

- Relationships configured via navigation properties: Employee.Leaves (ICollection<Leave>) and Leave.Employee (Employee?)
- Connection string managed via appsettings.json with the key DefaultConnection
- Code First migrations produce the full schema including foreign keys and indexes

5.3 ADO.NET

EmployeeAdoRepository in HRManagement.Infrastructure/ADO/ implements IEmployeeRepository using raw ADO.NET (SqlConnection, SqlCommand, SqlDataReader). This demonstrates direct SQL execution alongside EF Core:

- Uses parameterized queries (SqlParameter) to prevent SQL injection
- Manually maps SqlDataReader columns to Employee entity properties
- Registered in DI as an alternative to EmployeeRepository for demonstrating both patterns

6. US5–US7: Web Application

6.1 ASP.NET Core MVC

HRManagement.Web implements the MVC pattern with `EmployeeController` handling CRUD operations:

- Index action — lists all employees fetched via `EmployeeService`, rendered in `Views/Employee/Index.cshtml`
- Create action — GET renders form, POST validates `ModelState` and calls `AddEmployeeAsync`
- Edit action — GET loads employee by `EmployeeId`, POST updates via `UpdateEmployeeAsync`
- Delete action — confirms and calls `DeleteEmployeeAsync`

The default MVC route is configured as `{controller=Employee}/{action=Index}/{id?}`, making the employee list the landing page.

6.2 Razor Pages

The Dashboard section is implemented as a Razor Page in

`HRManagement.Web/Pages/Dashboard/Index.cshtml` with a code-behind `PageModel`. Razor Pages are used for self-contained pages that do not fit the controller-view pattern, demonstrating the framework's dual-model support.

6.3 Server-Side & Client-Side Validation

- Server-side: `DataAnnotations` (`[Required]`, `[EmailAddress]`, `[StringLength]`) on `ViewModels` and `DTOs`, validated via `ModelState.IsValid` in controllers
- Client-side: ASP.NET Core tag helpers generate HTML5 validation attributes; jQuery Unobtrusive Validation provides instant feedback without page reload
- Both layers are active simultaneously — client validation improves UX, server validation ensures security

6.4 AJAX Integration

AJAX calls are implemented in the Dashboard and employee views using the Fetch API to consume the REST endpoints without full page reloads, demonstrating the connection between the MVC web layer and the API layer within the same solution.

6.5 Routing

- Conventional routing: `MapControllerRoute` in `Program.cs` with the default pattern
- Attribute routing: `[Route("api/[controller]")]` on all API controllers
- Razor Pages routing: `MapRazorPages()` maps pages by file path convention

7. US8–US9: Web API & Security

7.1 RESTful API Surface

The API project exposes four controller groups documented in Swagger at </swagger/index.html>:

- Auth — POST `/api/Auth/login` (public — issues JWT token)
- Employee — GET, POST, PUT, DELETE `/api/Employee` (role-protected)
- Leave — GET, POST `/api/Leave` (role-protected)
- Analytics — GET `/api/Analytics/employee-count` (Admin only)

All endpoints follow REST conventions: POST returns 201 Created, successful DELETE/PUT returns 204 No Content, missing resources return 404 with ProblemDetails, unauthorized access returns 401/403.

7.2 JWT Authentication

`AuthService.GenerateToken()` builds a JWT with three claims:

- `ClaimTypes.Name` — the username
- `ClaimTypes.Email` — the user email
- `ClaimTypes.Role` — the role string (Admin / Manager / Employee)

The token is signed with HMAC-SHA256 using a secret key from `appsettings.json` (`Jwt:Key`), and carries configurable expiry via `Jwt:DurationInMinutes`. Token validation in `Program.cs` enforces issuer, audience, lifetime, and signing key.

7.3 Role-Based Authorization Matrix

Endpoint	Admin	Manager	Employee	Notes
GET /api/Employee	✓ 200 OK	✓ 200 OK	X 403	Authenticated users
GET /api/Employee/{id}	✓ 200 OK	X 403	✓ 200 OK	Admin + Employee
POST /api/Employee	✓ 201	X 403	X 403	Admin only
PUT /api/Employee/{id}	✓ 204	X 403	X 403	Admin only
DELETE /api/Employee	✓ 204	X 403	X 403	Admin only
POST /api/Leave	✓ 201	✓ 201	✓ 201	All roles
GET /api/Leave	✓ 200 OK	✓ 200 OK	X 403	Admin + Manager
POST /api/Auth/login	✓ Token	✓ Token	✓ Token	Public

Role enforcement is verified by logging in as each user type, authorizing in Swagger, and confirming the HTTP response code matches the matrix above. Employee tokens receive 403 on Admin-only endpoints; no token receives 401 on all protected endpoints.

7.4 Secure Coding Practices

Security Feature	Implementation	Effect
JWT Bearer Authentication	Token issued on login with Name, Email, Role claims	All API endpoints return 401 without valid token
Role-Based Authorization	[Authorize(Roles="Admin")], [Authorize(Roles="Admin,Manager")]	Employee cannot call Admin-only endpoints — returns 403
CORS Policy	WithOrigins() restricts to known origins only	Cross-origin requests from unknown domains blocked
HTTPS Redirection	app.UseHttpsRedirection() in pipeline	All HTTP traffic redirected to HTTPS
HSTS	app.UseHsts() enabled in production	Browser enforces HTTPS for subsequent requests
X-Content-Type-Options	nosniff header on all responses	Prevents MIME-type sniffing attacks
X-Frame-Options	DENY header on all responses	Prevents clickjacking via iframe embedding
ProblemDetails Error Format	ExceptionHandler returns RFC 7807 JSON	Stack traces never exposed in production
Input Validation	DataAnnotations on all DTOs + ModelState check	Invalid input rejected before hitting service layer

8. US10–US12: Cloud, DevOps & AI (Conceptual)

8.1 Docker Containerization Strategy (US10)

The HR Management Platform is designed for containerization with the following strategy:

- HRManagement.API — containerized as an ASP.NET Core runtime image (mcr.microsoft.com/dotnet/aspnet:8.0)
- HRManagement.Web — separate container for the MVC web application
- MS SQL Server — container using mcr.microsoft.com/mssql/server:2022-latest with a persistent volume mount for data
- Docker Compose orchestrates all three containers with an internal network, ensuring the API and Web containers communicate with SQL Server by container name
- Environment variables (connection strings, JWT keys) are injected at runtime via docker-compose.yml — never hardcoded in images

8.2 GitHub Actions CI/CD Pipeline (US11)

A GitHub Actions workflow (azure-deploy.yml) defines the continuous delivery pipeline:

- Trigger: push to main branch or pull request targeting main
- Build stage: dotnet restore → dotnet build --configuration Release
- Test stage: dotnet test — runs all xUnit tests; pipeline fails if any test fails
- Publish stage: dotnet publish -o ./publish for both API and Web projects
- Deploy stage: Azure Web App deploy action pushes the published artifact to Azure App Services
- Secrets (AZURE_CREDENTIALS, JWT_KEY, ConnectionStrings) stored in GitHub repository secrets — never in source code

8.3 Azure App Services Hosting Model (US11)

Component	Concept	Rationale
Azure App Service Plan	PaaS hosting for both API and Web projects as separate App Service instances	Managed infrastructure with auto-scaling and built-in HTTPS
Azure SQL Database	Managed SQL Server as a Service for production database	Eliminates DBA overhead; automatic backups and patching
Azure Key Vault	Stores JWT signing keys and connection strings as secrets	Removes all secrets from application configuration files
Azure Application Insights	Telemetry, request tracing, and exception monitoring	Provides the audit-readiness and observability the business requires

8.4 GitHub Copilot — AI-Assisted Development (US12)

GitHub Copilot was conceptually applied throughout the project for:

- Generating boilerplate repository implementations from interface definitions
- Auto-completing JWT token generation logic in AuthService
- Suggesting DataAnnotation attributes for DTO validation
- Proposing xUnit test method stubs from service method signatures

Copilot accelerates the mechanical parts of implementation, allowing the developer to focus on architectural decisions and business logic correctness.

8.5 AI Agent (MCP) Endpoint for Ticket Summarization (US12)

An AI Agent endpoint is conceptualized as POST /api/agent/summarize-leave-requests. When called, this endpoint would:

- Retrieve pending leave requests for the current period from LeaveService
- Format the data as a prompt and send to an LLM API (e.g., Claude claude-sonnet-4-20250514) via HTTP
- Return a natural-language summary for HR managers: 'This week, 8 leave requests are pending. 3 are from the Engineering department spanning more than 5 days.'
- The endpoint would be Admin/Manager-only, JWT-protected, following the same RBAC pattern as all other API endpoints

9. US13: Architecture & Documentation

9.1 Logical Architecture

The platform follows a strict Clean Architecture layering:

- Client Layer: Browser → HRManagement.Web (MVC/Razor) or Swagger UI / external clients
- API Layer: HRManagement.API — REST controllers, JWT middleware, exception middleware, security headers middleware
- Application Layer: HRManagement.Application — services encapsulating all business rules
- Domain Layer: HRManagement.Core — entities, interfaces, DTOs, custom exceptions (no dependencies)
- Infrastructure Layer: HRManagement.Infrastructure — EF Core DbContext, repositories, ADO.NET implementation
- Database: MS SQL Server — tables, relationships, triggers, indexes

9.2 GitHub Repository Structure

- Root: HRManagement.sln solution file
- src/HRManagement.Core — domain layer (no external NuGet dependencies)
- src/HRManagement.Application — application services
- src/HRManagement.Infrastructure — EF Core, repositories
- src/HRManagement.API — Web API project
- src/HRManagement.Web — MVC web project
- tests/HRManagement.Tests — xUnit test project
- docs/ — architecture diagrams, this report, database schema
- .github/workflows/azure-deploy.yml — CI/CD pipeline definition

9.3 Code Documentation Standards

- All public classes and interfaces have XML doc comments (///) describing purpose, parameters, and return values
- Controller actions document the HTTP method, route, required role, and response codes
- Services document business rules and exception conditions
- Program.cs is organized with comment banners separating each configuration section

9.4 Final Walkthrough Checklist

- Build: Rebuild All — 6 succeeded, 0 failed, 0 skipped
- Database: All tables present in SSMS; test users inserted with Admin, Manager, Employee roles
- Swagger: All endpoints visible with lock icons; Bearer auth functional
- JWT: Token decoded at jwt.io shows correct Name, Email, Role claims
- RBAC: Employee token returns 403 on Admin endpoints; Admin token returns 200
- DTOs: Swagger Schemas section shows EmployeeRequestDto, EmployeeResponseDto (not raw entity)
- ProblemDetails: Invalid ID returns application/problem+json with no stack trace
- Security headers: X-Frame-Options: DENY and X-Content-Type-Options: nosniff visible in DevTools
- MVC: Employee CRUD functional via web UI at localhost:7200
- Unit tests: All tests pass in Test Explorer

10. Conclusion

The HR Management Platform capstone successfully demonstrates end-to-end enterprise application development across 13 User Stories. The implemented system provides:

- A secure, role-gated REST API with JWT authentication verified in Swagger
- A clean six-project architecture following SOLID principles and Clean Architecture
- Both EF Core and ADO.NET data access patterns within the same solution
- A fully functional ASP.NET Core MVC web application with Razor Pages and AJAX
- Production-ready secure coding practices including CORS, HSTS, security headers, and ProblemDetails error handling
- A conceptual cloud deployment strategy on Azure with Docker and GitHub Actions CI/CD
- A conceptual AI modernization path using GitHub Copilot and an MCP agent endpoint

The platform is audit-ready, extensible, and built to production engineering standards — demonstrating readiness for SME-level technical responsibilities at Wipro.